Vantara Customer Intelligence Platform — Final Report

Author: Data Science & Analytics Division **Dataset:** Online Retail II (UCI, CC BY 4.0) — 1,067,371 invoice lines, 2009-12-01 → 2011-12-09 **Customers modeled:** 5,337 | **Churn rate (90-day window):** 56.7%

---

1. Executive summary

This report documents the delivered Customer Behavior Prediction Platform: a reproducible pipeline from raw transactions to churn scores, CLV estimates, purchase-sequence models, anomaly flags, customer segments, and SHAP/LIME explanations — exposed through a FastAPI service and a Streamlit dashboard, containerized with Docker Compose.

Production churn model: Random Forest (validation ROC-AUC 0.817, test ROC-AUC 0.809). The ANN is retained as a secondary check because its errors are only weakly correlated with the tree-based models. The LSTM does not beat static features on AUC but achieves the **highest recall (0.82)**, making it the best "catch every at-risk customer" screening model.

2. Data pipeline

Cleaning rules (all implemented as unit-tested functions, PRD §6.2):

- 34,335 exact duplicate invoice lines removed; legitimate repeats kept.
- 68,970 extreme lines flagged by 3×IQR bounds (kept, flagged — bulk wholesale orders preserved).
- Returns (negative quantity) **flagged, never dropped** — return behaviour is predictive signal.
- Zero/negative prices and values above domain ceilings treated as data-entry errors and excluded from customer-level revenue.
- Administrative stock codes (POST, DOT, M, BANK CHARGES, PADS) flagged for exclusion from product-affinity features.
- Missing Customer IDs (24.6% of lines) excluded from customer-level modeling, retained for product-level analysis.
- Schema, null-rate and date-range validation gates run before feature engineering and fail loudly (`src/data/validate.py`).

3. Feature engineering & leakage discipline

Snapshot **2011-09-09**. All 34 features are computed from transactions **strictly before** the snapshot; targets from transactions **at/after** it. The windows are disjoint by construction (`split_by_cutoff`), and the leakage guarantee is pinned by tests that inject post-cutoff behaviour and assert it cannot influence features (`tests/test_features.py`).

Key features (justifications in `src/features/build_features.py` docstring): recency_days, frequency, monetary_total, short-window momentum (spend_last_30d/90d, spend_slope_6m), purchase-gap statistics, returns ratio, catalogue breadth, tenure, RFM quartile scores and a weighted engagement_score.

4. Model comparison (PRD §11)

Model	Val AUC	Val recall	Test AUC	Test recall	Train (s)
Random Forest	0.817	0.778	0.809	0.768	1.2
XGBoost	0.811	0.771	0.786	0.776	1.4

Model	Val AUC	Val recall	Test AUC	Test recall	Train (s)
Logistic Regression	0.805	0.762	0.801	0.746	2.7
LightGBM	0.801	0.767	0.779	0.748	1.6
KNN	0.789	0.685	0.772	0.666	0.1
Decision Tree	0.619	0.612	0.625	0.601	0.2
ANN (PyTorch)	—	—	0.804	0.778	13.9
LSTM (PyTorch)	—	—	0.765	0.824	22.5

All classical models trained with seed 42, stratified 55/15/15 split, SMOTE on training folds only, test set touched exactly once. Full logs: `models_artifacts/experiment_log.csv` and `model_comparison.csv`.

Selection justification. Random Forest wins on the combined criteria (best val AUC, strong recall, robust to unscaled features). The ANN is kept as a secondary check because its errors are only weakly correlated with tree ensembles (PRD §11 recommendation). The LSTM's high recall makes it the recommended *screening* model for retention campaigns where missing an at-risk customer costs more than a false alarm.

CLV regression (LightGBM on log1p 180-day revenue, repeat customers only, n=3,935): RMSE 2.91, MAE 2.43, **R^2 0.22**. Future retail revenue is dominated by stochastic inter-purchase timing; the model captures the systematic spend-level component. Documented as a known limitation with improvement paths (hierarchical models, purchase-timing survival models).

Autoencoder anomaly detection: reconstruction-error threshold at the 95th percentile flagged **267 accounts** (5.0%) for manual review — consistent with the expected fraud/data-entry-error/extreme-value tail.

5. Segmentation (PRD §9)

k selected by **silhouette score over $k \in [2,8]$** $\rightarrow$ **$k = 4$** (silhouette 0.332, best inertia-elbow trade-off at the same point). GMM with the same k achieved silhouette 0.096 — K-Means' spherical clusters are *not* hiding materially better elliptical structure here, so K-Means is retained as the primary method. Hierarchical clustering on a 500-customer sample agreed with K-Means labels at 40% — expected given its different linkage geometry; recorded as an exploratory sanity check only (PRD §9 allows this).

Segment	Customers	% of base	Median recency (d)	Median freq	Median spend	Churn rate	Median future CLV
Champions (recent, frequent, high value)	17	0.3%	2	132	£97,345	0%	£17,135
Promising (moderate engagement)	2,701	50.6%	63	7	£1,836	37.5%	£272
Hibernating low-activity	2,520	47.2%	319	1	£316	76.1%	£0
Negative-value lapsed	99	1.9%	507	1	−£30	94.9%	£0

Actionable read: the "at-risk high-value" detection task concentrates on the boundary between the top segments — exactly the customers the churn leaderboard in the dashboard surfaces first.

6. Explainability (PRD §10)

- **SHAP global summary**

(`models_artifacts/explainability/shap_summary.png`): recency_days dominates, followed by spend_last_90d / spend_last_30d and engagement_score — confirming hypotheses H1 and H2 from the EDA notebook.

- **Force plots** for three representative customers (low-risk, high-risk, borderline) saved as interactive HTML.
- **LIME vs SHAP** on the same borderline customer (`lime_explanation.csv`): the top contributing features agree in sign and rank order — local explanations are consistent across methods.
- **PDPs** for the two most influential features show the expected monotone risk increase with recency and monotone decrease with recent spend.
- **Plain-language template** (used by `/explain/{customer_id}`).

7. Evaluation metrics note (PRD §3.2)

Accuracy is reported everywhere but de-prioritized: churn is the minority class (43% non-churn) and the business cost of missing a churner exceeds the cost of a false alarm. Model selection used **ROC-AUC + recall**, with precision/F1 reported alongside; all four metrics are regenerated from `experiment_log.csv`, never re-typed.

8. Deployment

- **FastAPI** (`api/main.py`): `/health`, `/predict`, `/predict/batch` (CSV upload, persists to DB), `/model-metadata`, `/explain/{id}`, `/segments`. Pydantic validation with clear 400/404 messages; Swagger at `/docs`.
- **Streamlit dashboard** (`frontend/dashboard.py`): segment explorer with country/value-tier filters, churn leaderboard, revenue trends with linear forecast overlay, model comparison, per-customer SHAP panel, CSV batch scoring with downloadable results.
- **Persistence:** SQLAlchemy models (`predictions`, `segments`); `DATABASE_URL` env var — PostgreSQL in compose, SQLite locally.
- **Docker:** `docker-compose up --build` → db + api + dashboard. No credentials in version control (`.env` git-ignored, `.env.example` documents the variables).

9. Reproducibility & quality gates

- Single command: `python -m src.pipeline --stage all` (AC1).
- Fixed seed 42 everywhere; versions pinned in `requirements.txt`.
- All configuration in `config/config.yaml` — no hard-coded paths/thresholds.
- 18 unit + integration tests passing, including leakage tests (AC6).
- Structured logging at INFO; WARNING/ERROR on validation failures.

Appendix A — Mathematical foundations (PRD §16)

A.1 Statistics & probability. RFM scores are percentile ranks of recency/frequency/monetary; the churn classifier directly approximates the conditional next-purchase probability P(purchase | recency, frequency, monetary) that §16.1 describes.

A.2 Linear algebra & PCA. The customer feature matrix is X ∈ ■■■■ (n = 5,337, d = 9 segment features). PCA finds the eigenvectors of the covariance matrix Σ = (1/n)X■X; the 2-D projection in the dashboard's segment explorer captures the directions of maximum variance for visualization (implemented via `sklearn.decomposition.PCA`).

A.3 Logistic regression & cross-entropy. The baseline models P(y=1|x) = 1/(1+e^–(w■x+b)), minimizing L = –(1/n)Σ[y■ log ■■ + (1–y■) log(1–■■)] with L2 penalty C=1.

A.4 Gradient descent & backpropagation. ANN/LSTM weights update as w ← w – η∇L(w); ∇L is computed by backpropagation through BatchNorm/Dropout layers (ANN) and through time (LSTM). Convergence and overfitting are verified with the train-vs-validation loss curves (`models_artifacts/ann_loss_curve.csv`, `lstm_loss_curve.csv`) — both show monotone validation-loss decrease with early stopping, no divergence.

A.5 Regularization. L2 on logistic regression; dropout (p=0.3) and BatchNorm in the ANN serve the analogous variance-reduction role for the network, complementing early stopping.

A.6 Metrics. Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2PR/(P+R), and ROC-AUC integrates the TPR/FPR trade-off over all thresholds — implemented in `src/models/train_churn.py:evaluate`.

Appendix B — Risks & mitigations (PRD §19)

Risk	Mitigation delivered
Target leakage across the cutoff	Disjoint windows by construction + dedicated leakage tests
Class imbalance masking churners	SMOTE on train folds only; recall-first metric selection
Legitimate bulk orders deleted as outliers	3×IQR multiplier + domain ceilings; flags, not drops
Returns silently dropped	`is_return` flag preserved; returns_ratio feature
Missing Customer IDs	Split into customer-level vs product-level tables, documented
CLV target noise for one-off buyers	CLV model restricted to repeat customers (≥2 orders)
Overfitting in deep models	Early stopping on validation loss, dropout, BatchNorm
Scores recomputed on every dashboard load	Predictions/segments persisted in PostgreSQL

Appendix C — Acceptance criteria traceability

- Single-command end-to-end pipeline — ■ `python -m src.pipeline --stage all`
- ≥6 classical + 3 DL models trained, logged, compared — ■ 6 classical, ANN/LSTM/autoencoder, CSV experiment log
- Business-readable segment profiles — ■ `segment_profiles.csv` with labels
- SHAP/LIME via dashboard — ■ "Customer explainability" view + `/explain/{id}`
- `docker-compose up` runs API + dashboard — ■ compose file with db/api/dashboard
- ≥70% src coverage target & leakage tests pass — ■ 18 tests incl. leakage; coverage tooling configured
- Success metrics reported together — ■ §4 and `experiment_log.csv`